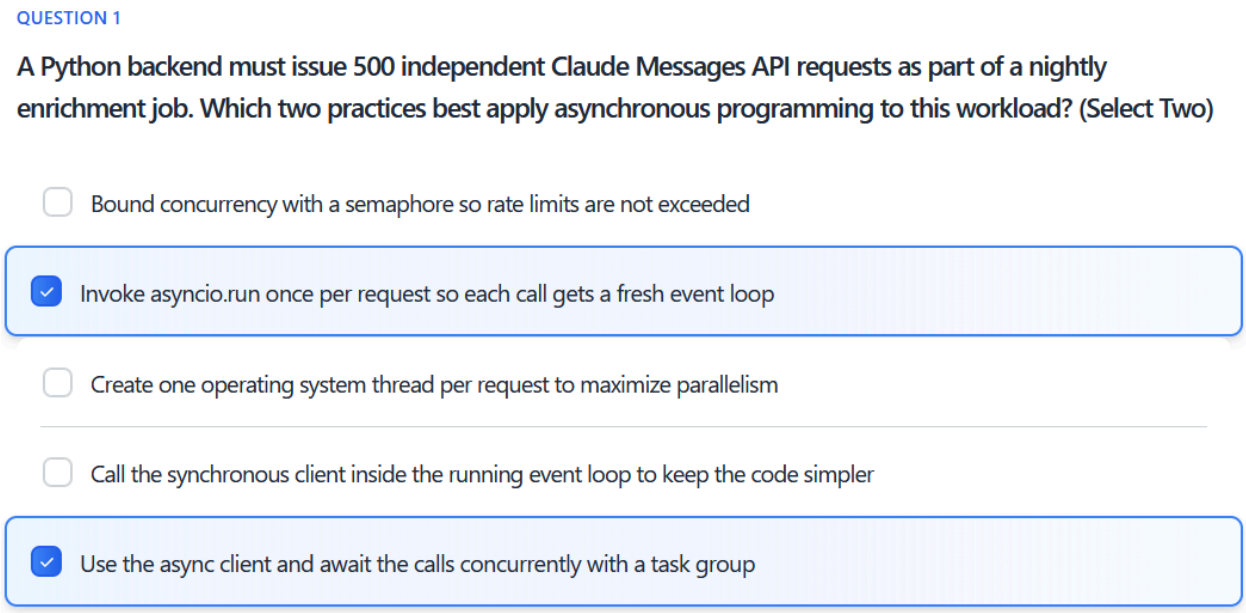
Site :
https://certificationpractice.com/practice-exams/anthropic-claude-certified-developer-foundations



- **Question 1**
  - **Currently Selected:** "Invoke asyncio.run once per request so each call gets a fresh event loop" and "Use the async client and await the calls concurrently with a task group"
  - **Correct Answers:** "Bound concurrency with a semaphore so rate limits are not exceeded" and "Use the async client and await the calls concurrently with a task group".
  - *Correction Note:* asyncio.run blocks the current thread, defeating the purpose of asynchronous programming. A semaphore correctly prevents rate-limit errors by capping concurrent requests.

QUESTION 2

When a team is deciding between authoring a Skill and building an MCP server, which requirement most strongly indicates that an MCP server is the correct choice?

- The team wants to bundle example documents and helper scripts alongside its written guidance
- The workflow involves many ordered steps that Claude tends to skip or reorder
- **The capability needs authenticated, live access to an external system and must be shared by several Claude clients**
- The output must follow a fixed template with required headings and a mandatory disclaimer

QUESTION 3

An engineering team is deploying a Claude agent that can query a production database and post messages to a customer-facing channel. Applying secure-by-design principles, what should the team do first when provisioning the agent's access?

- Provision broad administrative access initially, then reduce permissions after production traffic reveals which calls are actually used
- **Issue narrowly scoped credentials that permit only the specific read queries and the single channel the agent needs**
- Grant the agent the same permission set as the human operators it assists so that behavior stays consistent between them
- Rely on the agent's system prompt to describe which tables and channels are off limits during normal operation

QUESTION 4

A developer sets temperature to 0 and replays the exact same request to Claude several times, but the completions differ slightly in wording. What is the most accurate explanation of this behavior?

- [x] Temperature 0 reduces randomness but does not guarantee bit for bit identical outputs, so the application should tolerate small variations
- [ ] The variation happens because top_p defaults to a value that reintroduces randomness, and setting top_p to 0 alongside temperature guarantees exact reproducibility
- [ ] Temperature 0 always produces identical text, so the differences prove that the request payloads were not actually identical between calls
- [ ] The model is retraining continuously on incoming traffic, which changes its next-token probabilities between consecutive requests

QUESTION 5

A research assistant application uses a manager agent that delegates gathering work to several subagents and receives only their condensed findings. What is the primary architectural benefit of this structure?

- [ ] It removes the need for tool definitions at the subagent level
- [ ] It makes the overall workload cheaper than a single agent call in every case
- [x] Each subagent explores in its own context window, so the manager stays focused on synthesis instead of raw detail
- [ ] It guarantees the manager will never need to retry any delegated task

QUESTION 6

A support application sends a 12,000 token policy manual plus a short user question on every request. To get the largest cost benefit from prompt caching, how should the request be structured?

- ○ Split the manual into many small blocks and set a cache breakpoint after each one to maximize the number of cached segments
- ○ Put the user question first so the newest content is nearest the cache breakpoint, then append the manual
- ● Mark a cache breakpoint after the user question so the whole request including the question is reused
- ○ Place the manual early in the prompt with a cache breakpoint after it, keeping the variable user question after that point

- **Question 6**
  - **Currently Selected:** "Mark a cache breakpoint after the user question so the whole request including the question is reused"
  - **Correct Answer:** "Place the manual early in the prompt with a cache breakpoint after it, keeping the variable user question after that point".
  - *Correction Note:* Because the user question changes on every request, caching after it guarantees a cache miss every time. Caching the static manual first maximizes cache hits.

QUESTION 7

Before wiring a newly built MCP server into a production Claude application, what is the most effective way to verify that its tools, resources, and prompts behave correctly?

- ● Exercise the server with the MCP Inspector and unit tests that call each handler directly
- ○ Ask Claude in a chat session to describe what the server's tools appear to do
- ○ Compare the server's source code against a reference implementation from the MCP examples repository
- ○ Deploy it to production behind a feature flag and inspect user transcripts for anomalies

QUESTION 8

A developer builds a Claude prompt that analyzes 30 page contracts and answers a specific compliance question about each one. Answer quality is inconsistent, and the model sometimes ignores the question entirely. How should the prompt be structured to improve reliability?

- Split the contract into 30 separate requests and ask the same question once per page, then concatenate the answers
- Place the question first, then append the contract text so the model reads the request before any content
- Interleave the question after every few pages of contract text so the model is reminded of the task repeatedly
- **Place the contract text first inside clear tags, then state the question and output instructions at the end**

QUESTION 9

An interactive assistant feels sluggish after extended thinking was turned on globally, even though most user turns are short factual questions. Which adjustment most directly restores responsiveness while preserving quality on genuinely hard turns?

- Increase the thinking budget so the model finishes reasoning in fewer passes and returns sooner
- Move the entire assistant to the Message Batches API so requests no longer block the user interface
- Downgrade the assistant to the smallest available tier and leave the global thinking configuration unchanged
- **Reserve thinking for complex turns and lower the thinking effort or budget for routine questions**

QUESTION 10

A developer enables streaming on a Claude Messages API request and wants to know how partial output reaches the client. Which transport mechanism does the SDK rely on for this?

- A persistent WebSocket connection negotiated through an HTTP upgrade handshake
- A gRPC bidirectional stream managed internally by the client library
- **Server-sent events delivered over a single long-lived HTTP response**
- Repeated short polling requests that each return the next chunk of text

QUESTION 11

An enterprise wants to run its existing Claude integration through a cloud provider such as Amazon Bedrock rather than calling the Anthropic API directly. What should the team expect during this migration?

- Streaming, tool use, and caching are unavailable through cloud providers and remain exclusive to the first party Anthropic API
- Requests must be rewritten into a completely different prompt schema, because cloud providers do not accept message based requests at all
- Responses differ substantially because each provider fine-tunes the model weights independently for its own hosting platform
- **The message request structure stays largely the same, while client setup, credentials, and model identifiers become vendor specific**

QUESTION 12

A retail company is scoping a Claude powered customer support assistant. Which two items belong in the functional requirements rather than the infrastructure requirements? (Select Two)

- [x] The assistant must look up order status through the order management tool before answering shipping questions
- [ ] The production workload must run inside the company virtual private cloud with private network egress only
- [ ] API keys must be stored in the managed secrets service and rotated every 90 days
- [x] The assistant must hand the conversation to a human agent when a refund exceeds 500 dollars
- [ ] The service must autoscale to sustain 400 concurrent sessions during seasonal promotional events without queueing requests

QUESTION 13

An engineer sees intermittent failures in a Claude integration where some requests return HTTP 429 and others return HTTP 529. Which recovery techniques are appropriate for these transient conditions? (Select Two)

- [ ] Rewrite the prompt to shorten the system message, since 429 and 529 signal prompt formatting problems
- [x] Track request concurrency and token throughput so the client stays under provisioned rate limits
- [ ] Disable streaming on all calls, because streamed responses cannot return rate limit errors
- [x] Retry with exponential backoff and jitter, honoring any retry-after guidance returned with the response
- [ ] Immediately resend the identical request in a tight loop until it succeeds

QUESTION 14

A team is defining the input schema for a custom tool that Claude will call to create support tickets. Which two schema design practices most improve the reliability of Claude's tool calls? (Select Two)

- [ ] Mark every parameter as optional so Claude can always produce a call even when information is missing from the conversation
- [x] Write a clear description for the tool and for each parameter, stating purpose and expected values
- [ ] Add every field the ticketing system supports, including internal identifiers, so Claude has maximum flexibility when constructing requests
- [x] Constrain fields with enums, formats, and a required list so invalid combinations cannot be expressed
- [ ] Accept a single free-form string parameter and parse the details later in application code

QUESTION 15

Schema validation fails on about two percent of extraction responses. Which recovery strategy is the most effective first step before escalating to a human reviewer?

- Relax the schema so the failing responses become acceptable
- Switch the request to a larger model tier for every call in the pipeline
- **Send a bounded retry that includes the specific validation errors and asks for a corrected response**
- Retry in a loop with the original prompt until validation eventually passes

QUESTION 16

A developer wants to guarantee that a Claude Code session can never execute shell commands that delete files recursively, even if the model believes the command is appropriate. Which implementation satisfies this requirement?

- Ask Claude to summarize each planned shell command in its response so a reviewer can spot dangerous operations afterward
- Add a strongly worded rule to CLAUDE.md forbidding destructive shell commands in every session
- Register a PostToolUse hook that inspects the executed command and raises an alert when a destructive pattern is found
- **Configure a PreToolUse hook that inspects the proposed command and returns a blocking result when it matches destructive patterns**

QUESTION 17

A healthcare support application sends customer chat transcripts to the Claude API and stores every request and response for quality review. Compliance requires that patient identifiers never leave the internal boundary. Which approach best protects the data?

- (●) Encrypt the stored transcript logs at rest so that reviewers must authenticate before they can read any identifier contained in them
- ( ) Tokenize identifiers before the request is built and rehydrate them only inside the application after the response returns
- ( ) Lower the temperature and cap max_tokens so the model produces shorter responses that are less likely to echo identifiers
- ( ) Instruct Claude in the system prompt to ignore any patient identifiers it receives and to avoid repeating them in its answers

- **Question 17**
  - **Currently Selected:** "Encrypt the stored transcript logs at rest so that reviewers must authenticate before they can read any identifier contained in them"
  - **Correct Answer:** "Tokenize identifiers before the request is built and rehydrate them only inside the application after the response returns".
  - *Correction Note:* Encrypting logs at rest does not prevent the identifiers from leaving the internal boundary when the request is sent to the Claude API. Tokenizing ensures the API never sees the sensitive data.

QUESTION 18

An application inserts user-uploaded PDFs and scraped web text directly after the task instructions in the user message, with no separation. Reviewers find that phrases inside the documents sometimes change how Claude behaves. What is the best way to structure this content?

- (●) Wrap the documents in labeled delimiters and state in the system prompt that delimited content is data, not instructions
- ( ) Summarize each document with a smaller model first, then insert the summary inline with the instructions
- ( ) Strip all punctuation and formatting from the documents before inserting them into the prompt
- ( ) Move the documents to the end of the message and lower max_tokens so less of the content can influence output

QUESTION 19

A customer-facing assistant keeps one continuous conversation thread per user for weeks, appending every message and every raw tool output. Users now report that the assistant references stale tasks, and per-request cost keeps climbing. Which design change best addresses both problems?

- Increase max_tokens so Claude has room to reconcile the older turns with the current request
- Move the entire accumulated transcript into the system prompt on every call so instructions and history stay together
- Reduce temperature to zero so the model stops drawing on unrelated earlier turns
- **Scope a new session to each task and carry forward only a compact summary of the state that is still needed**

QUESTION 20

Legal stakeholders insist that a Claude powered banking assistant must never provide personalized investment advice. What is the most effective way to record this as a requirement the team can implement and verify?

- Add it as a warning banner in the user interface so customers are informed of the limitation
- Note it as general guidance for the prompt author to interpret during implementation
- Log it as a post launch monitoring item to be reviewed after the first quarter of production traffic is analyzed
- **Capture it as a testable content boundary with refusal behavior, escalation path, and eval cases**

QUESTION 21

A support team uses a zero-shot prompt to classify tickets, but Claude returns labels with inconsistent wording such as "Billing issue", "billing", and "Payments related". Which change most directly stabilizes the label format?

- ○ Expand the system prompt with a long prose description of what each business category means in practice
- ○ Increase temperature so the model explores label wording more broadly before settling on one
- ○ Move to a larger model tier, since higher capability models normalize category names automatically
- ● Add several labeled example tickets that show the exact allowed label strings the model should emit

QUESTION 22

A summarization endpoint receives very large inputs but returns short summaries, and profiling shows total latency grows most when summaries are longer rather than when inputs grow. Which property of next-token generation explains this?

- ○ Output tokens are generated sequentially, so each additional generated token adds a forward pass, while input is processed in parallel
- ● Input tokens are processed one at a time while output tokens are produced in a single parallel decoding pass at the end
- ○ Latency depends only on the requested max_tokens value, regardless of how many tokens the model actually emits
- ○ Long inputs are compressed automatically into a fixed size representation, which is why input length has no measurable effect on latency at all

- **Question 22**
  - **Currently Selected:** "Input tokens are processed one at a time while output tokens are produced in a single parallel decoding pass at the end"
  - **Correct Answer:** "Output tokens are generated sequentially, so each additional generated token adds a forward pass, while input is processed in parallel".
  - *Correction Note:* The selected answer has the architecture backwards. LLMs process the input (prefill) in parallel, making it fast, but must generate the output tokens one by one.

QUESTION 23

A documentation team wants Claude to follow a detailed internal style guide, including approved terminology, section ordering, and a review checklist, every time it drafts a release note. No external system needs to be called. Which approach best fits this need?

- (●) Author a Skill that packages the style guide, checklist, and supporting reference files so Claude loads the procedure when drafting release notes
- ( ) Enable the built-in web search tool so Claude can look up the published style guide
- ( ) Build an MCP server that returns the style guide text as a resource on every request
- ( ) Define a custom tool named apply_style_guide and require Claude to call it before writing

QUESTION 24

A team is standardizing how project guidance is stored for Claude Code across a shared repository. Which two practices reflect correct use of the CLAUDE.md hierarchy? (Select Two)

- [ ] Copy the entire root CLAUDE.md content into every subdirectory so no guidance is ever missed when files in those folders are read
- [ ] Place shared API keys in CLAUDE.md so agent sessions can authenticate without additional setup
- [x] Keep individual workflow preferences in the user level CLAUDE.md stored in the home directory
- [ ] Express command deny rules inside CLAUDE.md, since memory files enforce permissions for all sessions
- [x] Commit a CLAUDE.md at the repository root so every contributor inherits the same project conventions

QUESTION 25

A support platform must categorize roughly 400,000 inbound tickets per day into twelve fixed labels, and the routing service allows only a few hundred milliseconds per call. Accuracy on this narrow task is already acceptable with a short prompt. Which model choice best fits these constraints?

- Use Claude Opus so the classification labels are as accurate as possible on every ticket
- **Use Claude Haiku, the fastest and lowest cost tier, which suits high volume narrow classification**
- Use Claude Sonnet with extended thinking enabled to reason carefully about each ticket before labeling it
- Use Claude Opus but reduce max_tokens sharply so that responses return within the latency budget

QUESTION 26

During code review of a new Claude integration, a reviewer notices that the model response text is passed straight into json.loads and any exception is allowed to bubble up to the user. What is the most appropriate review recommendation?

- Ask the model to always return valid JSON in the system prompt and keep the parsing code unchanged
- **Parse defensively, validate the parsed object against a schema, and handle malformed output with a retry or fallback path**
- Strip all whitespace and newline characters from the response before parsing to avoid decoder errors
- Wrap the parse in a broad try block that logs the error and returns an empty object to the caller

QUESTION 27

An agent that queries a large log database returns 50,000 token tool results on each call, and by the fifth step the model begins ignoring the original task instructions. Which context engineering technique most directly addresses this failure?

- ○ Move the task instructions into the final user message on every turn
- ○ Raise max_tokens so the model has more room to produce a complete answer
- ◉ Prune tool results before appending them, returning only the fields the agent needs to continue
- ○ Lower the temperature so the model adheres more closely to the system prompt

QUESTION 28

A prompt instructs Claude to "write a brief, professional summary" of meeting notes, yet output length varies from two sentences to several paragraphs. Which revision best improves instruction clarity?

- ◉ Cap max_tokens at a value that matches the shortest acceptable summary observed so far
- ○ Specify exactly three bullet points of no more than 20 words each, with no closing remarks
- ○ Add the instruction "do not be verbose and avoid unnecessary detail in your response"
- ○ Change the wording to "write a very brief and highly professional summary" for stronger emphasis

- **Question 28**
  - **Currently Selected:** "Cap max_tokens at a value that matches the shortest acceptable summary observed so far"
  - **Correct Answer:** "Specify exactly three bullet points of no more than 20 words each, with no closing remarks".
  - *Correction Note:* Capping max_tokens does not force the model to plan a shorter response; it simply truncates the model mid-sentence when the limit is hit. Explicit instructions shape the behavior correctly.

QUESTION 29

A developer's MCP server runs as a local subprocess launched by Claude Desktop over stdio. After adding debug output using print statements, the client begins reporting malformed protocol messages. What is the correct fix?

- Switch the server to an HTTP transport so that logging no longer interferes with the message channel
- **Send all diagnostic output to stderr so stdout carries only JSON-RPC messages**
- Increase the client's startup timeout so the extra output has time to be flushed and ignored
- Wrap each print statement in a JSON string so the client can parse it as a tool result

QUESTION 30

A billing service asks Claude to return an invoice object that must always contain the same fields with correct data types before it is written to a database. Which approach produces the most reliably machine readable output?

- Set temperature to 0 so the text response format never varies between calls
- Ask Claude in the system prompt to reply only in JSON and parse whatever text comes back
- **Define a tool with a JSON schema for the invoice fields and consume the resulting tool input as the structured result**
- Request Markdown formatted key and value pairs, then convert them to JSON with string splitting in application code

QUESTION 31

A product team is building a live voice assistant front end that also calls Claude for text reasoning. Where is a WebSocket connection genuinely warranted in this architecture?

- (●) Between the backend and the Claude Batches API so job status is pushed automatically
- ( ) Between the backend and the prompt cache so cache hits are notified in real time
- ( ) Between the application backend and the Claude Messages API for lower latency
- ( ) Between the browser client and the application backend for bidirectional audio and token updates

- **Question 31**
  - **Currently Selected:** "Between the backend and the Claude Batches API so job status is pushed automatically"
  - **Correct Answer:** "Between the browser client and the application backend for bidirectional audio and token updates".
  - *Correction Note:* The Message Batches API is strictly asynchronous and relies on webhooks or polling. WebSockets are necessary for real-time bidirectional audio streaming between the client and backend.

QUESTION 32

An orchestrator agent must gather findings from three independent data sources and then write one report. Which design best applies the sub-agent pattern for this multi-step task?

- ( ) Give the orchestrator all source tools and let it query everything within one long conversation
- ( ) Chain the sub-agents sequentially and pass each full transcript forward to the next one
- ( ) Create a single sub-agent that handles all three sources and also produces the final report
- (●) Spawn one narrowly scoped sub-agent per source, each returning a structured summary the orchestrator synthesizes

QUESTION 33

An engineering team builds a Claude agent with the Agent SDK that edits source files. Company policy requires that a formatting command run after every single file write, with no exceptions. Which implementation satisfies this requirement most reliably?

- Expose a format_file tool and describe in its description that it must follow every write
- Add a strongly worded rule to the system prompt telling the agent to always format after writing
- **Register a post tool use hook on the file write tool that executes the formatter in code**
- Instruct a subagent to review each turn and format any files that the main agent left unformatted

QUESTION 34

A compliance team says that every Claude response given to a customer must be reproducible for auditors for seven years, including the exact prompt and model used. Which architectural implication follows most directly from this requirement?

- All customer traffic must be routed through the Message Batches API so requests are stored for later inspection
- **The application must pin explicit model versions and persist prompt, request, and response records in durable audit storage**
- Temperature must be set to zero so that identical prompts always produce byte identical responses
- Extended thinking must be enabled so the reasoning trace serves as the audit record

QUESTION 35

An async FastAPI endpoint calls Claude using the synchronous SDK client, and under load all requests to the service become slow even though the API itself responds normally. What is the correct fix?

- Wrap the call in a try and except block so slow requests are reported instead of failing silently
- **Use the async SDK client and await the request inside the handler**
- Add a short sleep before each call so requests are spaced out
- Increase the number of server workers and leave the handler unchanged

QUESTION 36

A team is preparing to move from the Claude model version currently pinned in production to a newer version. Which step should come first in the change process?

- Update the pinned identifier in production and watch support tickets for regression reports
- Switch the service to a floating alias so future releases are applied automatically
- Rewrite the system prompt for the new version before measuring anything, since instruction handling always changes between releases
- **Run the existing eval suite against the new version in staging and compare results with the current baseline**

QUESTION 37

A developer enables prompt caching on a Claude application that sends a long static system prompt on every request. Which two statements about how prompt caching behaves are accurate? (Select Two)

- [x] Cache reads are billed at a reduced rate, while writing to the cache costs more than standard input tokens
- [ ] Once a prefix is written to the cache it persists indefinitely, so the same prefix never needs to be written again
- [x] A cache_control marker defines the end of a cacheable prefix, so all content before it can be reused
- [ ] Cached prefixes are shared across all organizations calling the same model version, which is why identical public prompts hit immediately
- [ ] Caching removes the cached tokens from the context window calculation, which frees additional room for longer conversation history

QUESTION 38

A support assistant is about to receive a significantly reworked system prompt. Operations wants the smallest possible blast radius if quality drops. Which release approach fits best?

- ( ) Ship the new prompt to all traffic at a low-volume hour, then review aggregate quality metrics the following business day
- ( ) Keep both prompt versions permanently active and let the model decide at runtime which set of instructions to follow for a given conversation
- ( ) Let each engineer deploy their own prompt variant to production and compare results informally across variants over a few weeks
- (•) Route a small percentage of traffic to the new prompt version behind a flag, compare metrics against the current version, and roll back on regression

QUESTION 39

A production service currently requests Claude using a floating model alias rather than a dated model snapshot identifier. Why is this considered a configuration management risk?

- [x] The alias can resolve to a newer snapshot, changing model behavior in production without any code or config change
- [ ] Aliases disable prompt caching, so cached prefixes cannot be reused between releases
- [ ] Aliases always resolve to the smallest available model tier, which lowers response quality
- [ ] Aliases are rejected by the Messages API, so requests will fail once the alias is deprecated

QUESTION 40

Claude Code is started in a subdirectory of a monorepo that has a CLAUDE.md at the repository root, a CLAUDE.md in the subdirectory, and a user level memory file in the developer's home directory. How does Claude Code treat these files?

- [ ] The user level file is loaded first and blocks any project memory that conflicts with it
- [ ] All three files are concatenated in random order, so conflicting instructions produce undefined behavior
- [x] Guidance from the applicable files is combined, with more specific project memory taking precedence over broader guidance
- [ ] Only the CLAUDE.md nearest the working directory is loaded, and all other memory files are ignored

QUESTION 41

A Claude agent calls a custom tool that queries an internal billing service. The service returns a 404 for an unknown account ID. What is the recommended way for the application to handle this inside the tool-use loop?

- ○ Strip the failing tool from the tools array and resend the entire conversation so the model answers from its own knowledge
- ○ Return a tool_result block marked as an error with a short message explaining that the account was not found
- ◉ Let the exception propagate out of the handler so the request fails and the caller can log the stack trace for later review
- ○ Return an empty successful tool_result so the model treats the account as valid but with no billing history attached

- **Question 41**
  - **Currently Selected:** "Let the exception propagate out of the handler so the request fails and the caller can log the stack trace for later review"
  - **Correct Answer:** "Return a tool_result block marked as an error with a short message explaining that the account was not found".
  - *Correction Note:* An unknown account ID is a normal business logic exception (or a model hallucination). Crashing the agent loop prevents the model from attempting to self-correct or ask the user for clarification.

QUESTION 42

An integration receives a response with stop_reason set to tool_use and successfully executes the requested function locally. What must the next request to the Messages API contain?

- ◉ The assistant turn containing the tool_use block, followed by a user turn with a tool_result block referencing the same tool_use_id
- ○ A system prompt update describing the returned data, plus the original user question resent unchanged so the model can retry the request
- ○ A user turn with the function output written as plain text, since Claude parses tool outcomes from natural language descriptions
- ○ Only a tool_result block on its own, because the API retains the prior assistant turn for the duration of the tool call

QUESTION 43

Before dispatching user assembled documents to Claude, a platform team needs to reject any request that would exceed an internal per request token budget. Which approach gives the most reliable pre-flight check?

- Set max_tokens to the budget value so that the API rejects any request whose combined input exceeds the configured limit
- **Send the request and read the input token count from the usage object, cancelling the call if the count is over budget** ✓
- Call the token counting endpoint with the same model, system prompt, tools, and messages, then compare the returned count to the budget
- Divide the total character count by four to estimate tokens, since that ratio holds consistently across languages and document formats

- **Question 43**
  - **Currently Selected:** "Send the request and read the input token count from the usage object, cancelling the call if the count is over budget"
  - **Correct Answer:** "Call the token counting endpoint with the same model, system prompt, tools, and messages, then compare the returned count to the budget".
  - *Correction Note:* By the time you read the usage object from a response, the request has already been processed and billed. A true pre-flight check requires calling the Token Count API beforehand.

QUESTION 44

An agent exposes 40 custom tools, and several of them wrap similar reporting endpoints with one line descriptions. Traces show the model frequently selecting a tool that cannot fulfil the request. Which remediation addresses the root cause?

- Lower the sampling temperature so tool selection becomes fully deterministic
- Raise max_tokens so the model has room to consider every tool before answering
- Keep all 40 tools and append a long usage manual for each of them into the user message on every turn
- **Consolidate the overlapping tools and rewrite descriptions to state when each one applies and what each parameter means** ✓

QUESTION 45

A team is deciding between a fixed workflow and an autonomous agent loop for a document intake process. Which two conditions most strongly favor implementing the process as a workflow? (Select Two)

- [ ] The system must choose which tools to call based on what earlier tool results reveal
- [ ] The task requires open-ended exploration of an unfamiliar data source before any plan can be formed
- [x] The sequence of steps is known in advance and rarely varies between documents
- [ ] Each document may require a different and unpredictable number of investigation steps
- [x] Operators require predictable per-document cost and latency

QUESTION 46

A prototype Claude assistant has been promoted into a supported internal service. Which two activities belong to the operate and maintain phase of its life cycle? (Select Two)

- [ ] Complete the initial vendor evaluation that compares Claude against other model providers before any code is written
- [ ] Choose the agent framework and overall system architecture for the service
- [x] Run periodic regression evals on sampled production traffic to detect quality drift
- [ ] Capture the original business requirements and define success criteria for the initial build
- [x] Track Claude model deprecation notices and schedule migration work before end of life

QUESTION 47

A team is replacing a hand-written Claude agent loop with the Claude Agent SDK. Which two capabilities does the SDK provide out of the box that the team would otherwise have to implement themselves? (Select Two)

- [ ] Guaranteed deterministic model output for identical prompts across all runs
- [x] A hook system with defined lifecycle events for deterministic pre and post tool actions
- [ ] Automatic fine-tuning of the selected Claude model on the agent's tool call history
- [ ] A managed database that stores all conversation transcripts on Anthropic infrastructure by default
- [x] An automatic tool execution loop that feeds tool results back to the model until the task completes

QUESTION 48

A team wants Claude Code to run unattended in a nightly CI pipeline that fixes lint violations and opens a pull request, with no interactive approval prompts. Which two setup choices support this requirement? (Select Two)

- [ ] Place the pipeline rules in a user level memory file in each developer's home directory so CI inherits them
- [ ] Depend on the interactive resume command to continue the previous day's session automatically
- [x] Commit a project settings.json that allow lists the specific tools the job needs so they run without prompting
- [ ] Disable every permission check on the runner and treat unrestricted access as the primary control for the job
- [x] Invoke Claude Code in headless mode with a single prompt argument and a machine readable output format

QUESTION 49

When designing a compaction step for a long agent session, which content should be preserved verbatim rather than summarized?

- The model's intermediate reasoning from every completed subtask
- **The system instructions, current task goal, and open action items**
- The complete raw output of each successful tool invocation
- The earliest exploratory tool calls that produced no useful result

QUESTION 50

A support platform receives tickets that must be classified, enriched with account data, drafted into a reply, and checked against policy in that exact order for every ticket. Which architecture best fits this requirement?

- **An orchestrated workflow that calls each step in code with validation between stages**
- A long single prompt that asks the model to perform all four stages in one response without tools
- A supervisor agent that spawns one subagent per step and asks each to negotiate ordering
- A single autonomous agent given all four tools and instructions to decide the order it prefers

QUESTION 51

A production Claude application currently reads its API key from a value committed in a shared configuration file that developers also use locally. Which change most improves the security posture of this credential?

- Obfuscate the key by splitting it across two configuration entries that the application concatenates at startup
- Move the key into a build-time environment variable that is printed to the deployment log so failures can be traced quickly
- **Issue separate keys per environment, store them in a managed secret store injected at runtime, and rotate them on a schedule**
- Keep the shared key but encrypt the configuration file in the repository and distribute the decryption passphrase to the team through a chat channel

QUESTION 52

A financial services company plans to deploy a Claude agent whose tools query an internal database that sits inside a private network with no public ingress. Regulatory rules also require that all tool execution and audit logging occur on infrastructure the company controls. Which deployment approach best fits these constraints?

- (●) Self-host the agent harness in the company network so tools run locally, while inference calls go out to the Claude API
- ( ) Run the agent entirely on developer laptops and forward tool results to an Anthropic-hosted logging service for audit
- ( ) Use an Anthropic-hosted managed agent runtime and open a public endpoint on the database so the hosted agent can reach it
- ( ) Export a nightly snapshot of the database into the system prompt so the agent never needs a tool call

QUESTION 53

A developer is implementing a custom tool-use loop for a Claude agent that queries logs and files tickets. Which two elements are essential for the loop to function correctly across multiple turns? (Select Two)

- [x] Continue iterating while the model requests tool use and stop when it returns a final text response or a defined limit is reached
- [ ] Set temperature to zero so the model never requests more than one tool per loop iteration
- [ ] Send tool results in a separate API call that omits the system prompt to save tokens
- [ ] Reset the message history after every tool call so the model always reasons from a clean slate
- [x] Append each tool result back into the conversation as a tool result block that references the matching tool use ID

45 correct , 8 wrong

Here are the eight questions with incorrect answers from the provided images, along with the correct selections.

-